

Code Explanation: Phase 2 - Data Preparation

Project: NeuroVision (Brain Tumor Classification) **Objective:** To preprocess the raw MRI images by resizing, converting them to numerical arrays, normalizing the values, and splitting them into training and testing datasets, aligning with the Unit III Lab syllabus.

1. Library Imports and Initial Setup

This section imports the required computer vision and machine learning libraries and initializes the variables.

- `import cv2`: Imports OpenCV (Open Source Computer Vision Library), a powerful tool used to read, resize, and manipulate image data.
- `import numpy as np`: Imports NumPy, a library used for handling large, multi-dimensional mathematical arrays and matrices.
- `from sklearn.model_selection import train_test_split`: Imports a specific function from scikit-learn used to divide our dataset into a training set and a testing set.
- `X = []`: Initializes an empty list to store the image data (features). In Machine Learning, 'X' typically represents the input variables.
- `y = []`: Initializes an empty list to store the labels (Tumor = 1, Normal = 0). 'y' represents the output/target variable.
- `IMG_SIZE = 128`: Sets a standard image size. All MRI scans will be resized to 128x128 pixels to ensure the neural network receives consistent input dimensions.

2. Processing 'Tumor' Images (Label = 1)

This loop processes all the images where a tumor is present.

- `for img_name in yes_images::` Starts a loop that goes through every single file name inside the 'yes' (Tumor) folder.
- `img_path = os.path.join(yes_path, img_name)`: Creates the exact file path for the current image.
- `img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)`: Reads the image file from the path and instantly converts it into a grayscale format (removing color channels).
- `if img is not None::` A safety check to ensure the image was read successfully and is not corrupted.
- `img_resized = cv2.resize(img, (IMG_SIZE, IMG_SIZE))`: Resizes the image to our standard 128x128 resolution.

- `X.append(img_resized)`: Adds the processed image array to our feature list X.
- `y.append(1)`: Adds the number 1 to our label list y, indicating this image contains a tumor.

3. Processing 'Normal' Images (Label = 0)

This loop does the exact same process for the healthy brain scans.

- `for img_name in no_images::` Loops through the 'no' (Normal) folder.
- The reading and resizing steps are identical to the 'yes' loop.
- `X.append(img_resized)`: Adds the normal image array to our feature list X.
- `y.append(0)`: Adds the number 0 to our label list y, indicating this is a healthy brain scan.

4. Data Transformation and Normalization

This block converts the Python lists into Deep Learning-compatible formats and normalizes the data.

- `X = np.array(X) & y = np.array(y)`: Converts our Python lists into NumPy arrays, which are significantly faster and required by deep learning frameworks like TensorFlow/Keras.
- `X = X / 255.0`: Performs **Normalization**. Pixel intensity values range from 0 (black) to 255 (white). Dividing by 255 scales all values down to a range between 0 and 1. This helps the neural network learn much faster and prevents exploding gradients.
- `X = X.reshape(-1, IMG_SIZE, IMG_SIZE, 1)`: Reshapes the array to explicitly include the "channel" dimension. 1 indicates a single grayscale channel (if it were an RGB image, this would be 3). The -1 tells NumPy to automatically figure out the total number of images.

5. Train-Test Split

This step divides the data so the model can be evaluated on unseen images.

- `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=42)`: Randomly shuffles and splits the dataset.
 - `test_size=0.20` means 20% of the images are kept aside for testing, while the remaining 80% will be used to train the model.
 - `random_state=42` ensures that the random shuffle is exactly the same every time we run the code, providing consistent results.

- `print(...)`: The final print statements output the total count and the exact shapes of our new training and testing arrays so we can verify the process worked correctly.